AMLNN Advanced Quantization User Guide

Revision: 0.1

Release Date: 2026-09-24

Copyright

© 2026 Amlogic. All rights reserved. No part of this document may be reproduced, transmitted, transcribed, or translated into any language in any form or by any means without the written permission of Amlogic.

Trademarks

Amlogic, and other Amlogic icons are trademarks of Amlogic companies. All other trademarks and registered trademarks are property of their respective holders.

Disclaimer

Amlogic may make improvements and/or changes in this document or in the product described in this document at any time and without notice.

This product is not intended for use in medical, life saving, or life sustaining applications, nor in applications where failure or malfunction of an Amlogic product can reasonably be expected to result in personal injury, death or severe property or environmental damage.

Circuit diagrams and other information relating to products of Amlogic are included as a means of illustrating typical applications. Consequently, complete information sufficient for production design is not necessarily given. Though every effort has been made to ensure that this document is current and accurate, Amlogic makes no representations or warranties with respect to the accuracy or completeness of the contents presented in this document.

Amlogic products may be subject to unidentified vulnerabilities or may support established security standards or specifications with known limitations. Amlogic accepts no liability for any vulnerability.

Use of the materials may require a license of intellectual property from a third party or from Amlogic. This document conveys no express or implied licenses to any intellectual property rights belonging to Amlogic or any other party. Any links to third-party websites included in this document are for your convenience only. Amlogic does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Amlogic shall in no event be liable for any claims relating to or arising out of this document or any use or inability to use hereof.

Contact Information

- Website: www.amlogic.com
- Pre-sales consultation: contact@amlogic.com
- Technical support: support@amlogic.com

Revision history

Version	Change Description
V0.1 (2026-09-24)	This is the First Version.

Contents

1 Document Description
- 1.1 Document Purpose
- 1.2 Supported Platforms
- 1.3 Keyword Description

2 Introduction to Quantization
- 2.1 Introduction to Quantization
  - 2.1.1 Quantization Definition and Mapping Ranges
  - 2.1.2 Quantization Parameter Calculation Formulas
- 2.2 Quantization Capabilities
- 2.3 Quantization Algorithms
  - 2.3.1 Feature Quantization Algorithms
  - 2.3.2 Weights Quantization Algorithms
  - 2.3.3 Smooth O3 Quantization Algorithm
- 2.4 Quantitative Strategy
  - 2.4.1 CV Model Quantization Guidance
  - 2.4.2 Transformer Model Quantization Guidance
  - 2.4.3 Quantization Guidance Summary

3 Quantization Conversion
- 3.1 Quantization Conversion Example
- 3.2 Model Format and Quantization Algorithm Compatibility
- 3.3 Model Accuracy Testing Under Different Quantization Granularities

4 Hybrid Quantization Guidance
- 4.1 Introduction to Mixed Quantization
- 4.2 Mixed Quantization Guidelines

5 quantize_optimization_tool Usage Guide
- 5.1 Tool Introduction
- 5.2 Usage Guide
- 5.3 Usage Recommendations
- 5.4 Custom Evaluation Criteria

6 hybrid_ratio_tool Usage Guide
- 6.1 Tool Introduction
- 6.2 YOLOX-S Model Validation
  - 6.2.1 W4A16 Quantization Validation
  - 6.2.2 w4a8 Quantization Validation
- 6.3 Usage Recommendations

1 Document Description

1.1 Document Purpose

This document serves as an advanced guide for quantization conversion on the Amlogic NPU platform, systematically describing the quantization algorithms applied to Weights and Feature Maps for the various quantization formats (Quantization Format) currently supported. It also focuses on the two quantization search tools, `hybrid_ratio_tool` and `quantize_optimization_tool`, guiding developers to leverage automated tool-based search to obtain the optimal balance between quantization accuracy and performance.

1.2 Supported Platforms

This guide applies to the following Amlogic NPU platforms:

- C308L / C302X
- S928X
- A311D2
- T968D4
- S905X5 / S905D5
- C302X2
- C305X2/A123X
- A311Y3

> C305X2/A123X and A311Y3 belong to **ADLA3**; the other platforms belong to **ADLA2**.

1.3 Keyword Description

- **ADLA2 (Second-generation Amlogic NPU chip)**: The hardware instruction set only supports fixed-point integer format operations such as int8, uint8, and int16; there is no native floating-point acceleration unit.
- **ADLA3 (Third-generation Amlogic NPU chip)**: On top of the fixed-point computation capability, adds native hardware support for fp16 and bf16 floating-point formats as well as int4 low-bit format operations.
- **Feature quantization (Feature Map quantization)**: The quantization operation performed on the feature map activation values generated during inference, using perlayer quantization granularity by default.
- **Weights quantization**: The quantization operation performed on the trainable weight parameters in a model.
- **Perchannel quantization**: Quantization performed with a single output channel as the independent unit, typically enabled for the weights of convolutional and fully-connected layers.
- **Perlayer quantization**: Quantization performed uniformly with the single network layer to which the complete tensor belongs as the overall unit.
- **Pergroup quantization (sub-channel group quantization)**: Further dividing the weights within a single channel into multiple independent groups and performing quantization with the sub-channel group as the unit, which can significantly reduce accuracy loss in lower-bit quantization scenarios.

2 Introduction to Quantization

2.1 Introduction to Quantization

Model quantization is an efficient model compression and deployment optimization technique that converts model parameters originally stored as 32-bit/64-bit high-precision floating-point numbers into 16-bit, 8-bit, or even lower 4-bit low-precision formats. With acceptable minor accuracy loss, it achieves order-of-magnitude reductions in model size and memory footprint, and greatly improves inference speed.

2.1.1 Quantization Definition and Mapping Ranges

The core of quantization is to map high-precision floating-point data into a low-precision integer range. The mapping ranges for different bit widths are as follows:

Quantization Type	Data Type	Mapping Range (Quantized Range)	Description
UInt8	Unsigned Int8	[0,255]	Unsigned 8-bit integer
Int8	Signed Int8	[−128,127]	Signed 8-bit integer
Int16	Signed Int16	[−32768,32767]	Signed 16-bit integer, for scenarios with high accuracy requirements

2.1.2 Quantization Parameter Calculation Formulas

The quantization process mainly involves two key parameters: **Scale** and **Zero Point**.

- **UInt8 / Int8 Quantization Formula (Asymmetric Quantization)**

  Applicable to scenarios where the data distribution is asymmetric or contains negative values, adjusting the zero point flexibly via the Zero Point.

  - **Scale calculation**:
    `scale = (float_max – float_min) / (Qmax – Qmin)`
  - **Zero Point calculation**:
    `zero_point = Qmax – float_max / scale`
    Note: The result usually needs to be rounded to an integer and clamped to the valid range of the corresponding integer type.

> **Symbol description**:
>
> - `float_max / float_min`: the maximum / minimum value of the floating-point data
> - `Qmax / Qmin`: the maximum / minimum integer value of the target quantization type (e.g., 127 / −128 for Int8)

- **Int16 Quantization Formula (Symmetric Quantization)**

  Typically adopts a symmetric quantization strategy, forcing the zero point to 0, which simplifies the computation logic and suits data with a large dynamic range and a relatively symmetric distribution.

  - **Intermediate variable calculation**:

    `float_max_temp = max(|float_max|, |float_min|)`

  - **Scale calculation**:
    `scale = float_max_temp / Qmax` (where `Qmax = 32767`)

  - **Zero Point**:

    `zero_point=0`

2.2 Quantization Capabilities

Comparison table of quantization capabilities across all Amlogic NPU platforms:

Platform Model	Feature (Feature Map) quantization	Weights quantization	Exclusive high-end capabilities
ADLA2	✅ int8 fixed-point quantization ✅ int16 fixed-point quantization	✅ int8 fixed-point quantization	• Only basic fixed-point quantization is supported
ADLA3	✅ int8 fixed-point quantization ✅ int16 fixed-point quantization ✅ fp16 / bf16 non-quantized mode	✅ int8 fixed-point quantization ✅ int4 low-bit quantization ✅ fp16 / bf16 non-quantized mode	• basic fixed-point quantization is supported • Supports low-bit large-model deployment and high-precision non-quantized inference

> **Special notes**:
>
> - Some hardware models do not directly correspond to the publicly named ADLA3 platforms such as A311Y3, C305X2, and A123X. Please refer to Section **2.1.3: Platform Type Confirmation** of the <02_Amlogic_NPU_User_Guide> document and complete the capability mapping via the `target_platform` field, to ensure that the quantization configuration fully matches the hardware capabilities and to avoid operator incompatibility, accuracy anomalies, and other issues.

Comparison table of supported quantization granularity:

Quantization Target	Data Precision	Supported Granularity	Applicable Operators / Description

Quantization Target	Data Precision	Supported Granularity	Applicable Operators / Description
Feature (Activation)	INT8 / INT16	perlayer	Per-layer quantization is uniformly supported on all platforms, i.e. all channels within the same layer share the same set of quantization parameters.
Weights	INT8	perlayer perchannel	• Per-layer: quantization parameters shared across the whole layer. • Per-channel: quantized independently along the output channel dimension, significantly improving the accuracy of Conv/FC operators.
Weights	INT4	perchannel pergroup	• Per-channel: quantized independently along the output channel dimension. • Per-group: group quantization is only supported for FC (fully-connected) operators, further reducing the memory footprint of large-model weights.

Detailed description:

- **Feature quantization limitation**: currently, feature map (Activation) quantization only supports the **perlayer** mode
- **Weights INT8 quantization**: supports both the **perlayer** and **perchannel** modes
- **Weights INT4 low-bit quantization**:
  - **Conv / DwConv operators**: 4-bit quantization uses the **perchannel** method
  - **FC operator**: 4-bit quantization uses the **per-group** method, with the group granularity configured by the group_size parameter.

2.3 Quantization Algorithms

2.3.1 Feature Quantization Algorithms

- **normal (Min-Max quantization, default)**
  - **Principle**: directly takes the global maximum and minimum values in the calibration dataset as the quantization boundaries.
  - **Characteristics**: simplest computation logic and lowest overhead.
  - **Applicable scenarios**: scenarios with a uniform data distribution and no obvious outliers.
- **percentile (percentile quantization)**
  - **Principle**: to prevent global extremes from being disturbed by outliers, uses a high quantile (e.g. 99.9%) and a low quantile (e.g. 0.1%) to clip the effective value range. Combines exponential moving average (EMA) to smooth the results of multiple calibration rounds.
  - **Characteristics**: can effectively suppress the impact of extreme outliers on the quantization range and improve quantization stability.

    - **Applicable scenarios**: scenarios where the activation values contain a few extreme outliers but the overall distribution is relatively concentrated.
- **histogram (histogram quantization)**
    - **Principle**: first builds a histogram of the absolute values of the activations to characterize the data distribution; then traverses the candidate threshold space, computing the distribution difference before and after quantization (e.g. KL divergence), and selects the per-round optimal threshold with the smallest difference; finally combines EMA smoothing over multiple rounds to obtain the final boundary.
    - **Characteristics**: optimizes the threshold based on data distribution similarity and can better preserve the statistical properties of the original data.
    - **Applicable scenarios**: activation quantization scenarios with complex, non-uniform data distributions and high requirements for distribution consistency (similar to the KLD strategy in TensorRT).
- **omse (Optimal Mean Squared Error)**
    - **Principle**: a search-based optimization scheme. Starting from the original data range `[raw_min, raw_max]`, it continuously shrinks the candidate interval `[new_min, new_max]` inward, traverses multiple candidate intervals and computes the overall mean squared error (MSE) before and after quantization, and finally selects the interval with the smallest MSE as the quantization boundary.
    - **Characteristics**: with minimizing reconstruction error as the goal, it usually achieves higher numerical restoration accuracy, but with relatively large computational overhead.
    - **Applicable scenarios**: scenarios extremely sensitive to quantization accuracy that allow higher calibration computation costs.

2.3.2 Weights Quantization Algorithms

- **normal (Min-Max quantization, default)**
    - **Principle**: directly takes the global maximum and minimum values in the calibration dataset as the quantization boundaries.
    - **Characteristics**: simplest computation logic and lowest overhead.
    - **Applicable scenarios**: scenarios with a uniform data distribution and no obvious outliers, such as model weight quantization.
- **lsq (Learned Step Size Quantization)**
    - **Principle**: a quantization parameter learning algorithm based on gradient backpropagation. It treats the quantization step size (Scale) as a trainable parameter, uses the straight-through estimator (STE) to resolve the gradient interruption caused by the rounding operation, and introduces a bit-width gradient scaling factor during backpropagation to balance the update magnitudes. In the quantization conversion stage, a layer-by-layer optimization strategy is usually adopted: other layers' parameters are fixed while the current layer's weights and quantization step size are fine-tuned individually.
    - **Characteristics**: by finding the optimal quantization parameters through end-to-end gradient descent, it can significantly reduce quantization error in extremely low-bit scenarios (such as W4A8) and maintain model accuracy.
    - **Applicable scenarios**: extremely low-bit weight quantization scenarios with extremely high accuracy requirements that allow a certain calibration computation cost.
- **decoupleQ**

- **Principle**: a weight quantization decoupling algorithm based on the mathematical optimization perspective. It abandons the traditional heuristic handling of outliers and models quantization as a constrained optimization problem. The core mechanism decouples the model weights into an "integer part" and a "floating-point residual part", solving for the optimal integer quantized values and floating-point compensation parameters separately through alternating optimization.
- **Characteristics**: the generated quantized weights have a linear uniform property that better suits the hardware acceleration engine; without introducing additional inference operator overhead, it achieves near-FP16/BF16 original-accuracy restoration.
- **Applicable scenarios**: low-bit (e.g. 4-bit) weight quantization scenarios, especially large-model deployments requiring a balance of high hardware efficiency and high accuracy

2.3.3 Smooth O3 Quantization Algorithm

Smooth O3 is an advanced post-training quantization (PTQ) scheme based on the classic SmoothQuant theory, deeply optimized for the Amlogic NPU hardware architecture. Its core goal is to solve the quantization accuracy collapse problem caused by extreme outliers in the activation values of large models and complex CV models, thereby achieving high-accuracy fixed-point quantization (e.g. W8A8) on key operators such as Conv2d, Depthwise Conv2d, and FC

1. Core Principle and Mechanism

The activation value distributions of Large Language Models (LLMs) and deep Transformer structures are often highly non-uniform, containing a few extremely large outliers. Direct quantization forces the quantization range to become too large, causing severe loss of representation accuracy for most ordinary values. Smooth O3 solves this problem with a "difficulty migration" strategy:

- **Activation smoothing**: divides the input activations by a perchannel smoothing factor, greatly compressing the magnitude of the outliers so that the activation distribution becomes more uniform and better fits the limited representation range of INT8/INT16.
- **Equivalent weight compensation**: to preserve mathematical equivalence, the same smoothing factor is multiplied into the weight matrix. Since the weight distribution is itself relatively smooth and highly redundant, this transformation does not significantly increase the weight quantization error.
- **Result**: the main difficulty of quantization is shifted from "hard-to-quantize activations" to "easy-to-quantize weights", minimizing the overall quantization error.

2. Graph Structure Changes

Operator insertion: the smoothing factor is not hardcoded or ignored; instead, it is automatically inserted in the form of an independent **Mul (multiplication) operator** at the front of the inputs of the target operators (Conv2d/FC, etc.).

3. Usage Guide

- **Configuration:** set the following parameter in the configuration interface of the quantization conversion script to enable it globally:

```
amlnn.config(apply_smooth_o3 = True)
```

- **Recommended enablement scenarios**

    - **Fully fixed-point quantization (W8A8)**: recommended. This is the scenario with the greatest benefit from Smooth O3; some models show significant accuracy improvement with INT8 weights combined with INT8 activations.
    - **Mixed-precision quantization (W8A16)**: recommended. Although the activations retain higher precision, the smoothing still optimizes the weight distribution and further improves overall robustness.
- **Measured benefit notes**
    - **Optimal-solution standard configuration**: most of the w8a8 quantization schemes verified internally have the Smooth O3 capability enabled.

2.4 Quantitative Strategy

Given the significant differences in quantization strategies between CV models and Transformer models, this chapter describes how to select the appropriate quantization approach.

To precisely evaluate the actual impact of different quantization strategies (e.g. INT8 Per-channel, mixed precision, etc.) on model accuracy, the amlnn-toolkit has a built-in fast verification mechanism based on standard formats: simply set `export_intermediate=True` in the configuration interface to directly export the intermediate TFLite model with quantization parameters already injected during the quantization conversion flow. Based on this model, you can directly run inference tests through TFLite Runtime to quickly measure the error difference before and after quantization and efficiently locate accuracy bottlenecks.

2.4.1 CV Model Quantization Guidance

For traditional convolutional neural networks (such as ResNet, YOLO, MobileNet, etc.), the activation distribution is relatively uniform and tolerance to quantization error is high. A "progressive" fully-integer quantization is recommended; it is recommended to follow a **progressive tuning path from low precision to high precision and from coarse granularity to fine granularity**, prioritizing extreme inference speed and memory compression.

Recommended tuning path:

1. **baseline:** use INT8 perlayer quantization with MinMax calibration as the initial scheme

```
amlnn.config(normalization_mean=[[128,128,128]],normalization_std=
[[128,128,128]], \
       quantized_dtype='w8a8', quantized_method='perlayer', \
       target_platform='PRODUCT_PID0XA007', export_intermediate=True)
```

    - **Expected:** most classic CV models can maintain very high accuracy with this configuration, and inference speed is the fastest.
2. **Step 1 (accuracy fine-tuning)**: if accuracy is insufficient, upgrade the weight quantization granularity to INT8 perchannel while keeping MinMax calibration.
    - **Principle:** the value distributions differ considerably across channels of a convolution kernel; perchannel can significantly reduce weight quantization error.

```
amlnn.config(normalization_mean=[[128,128,128]],normalization_std=
[[128,128,128]], \
    quantized_dtype='w8a8', quantized_method='perchannel',
apply_smooth_o3=True,\
    target_platform='PRODUCT_PID0XA007', export_intermediate=True)
```

3. **Step 2 (activation optimization)**: when accuracy still fails to meet the target, switch the activation calibration to percentile, histogram, or omse, and switch the weight calibration to lsq or decoupleQ (supported from the next tool version).
    - ***Principle*:** a CV model's activations may contain a few outliers; the percentile or histogram algorithm can determine the quantization boundary more robustly.
    - Refer to Chapter 5 of this document: quantize_optimization_tool usage guide; use this tool to quickly traverse combinations of key parameters such as calibration algorithms and quantization granularity, searching for the accuracy-optimal solution of W8A8 quantization.

```
amlnn.config(normalization_mean=[[128,128,128]],normalization_std=
[[128,128,128]], \
    quantized_dtype='w8a8', quantized_method='perchannel',
apply_smooth_o3=True, \
    activation_quant_algo="histogram",weight_quant_algo="lsq", \
    target_platform='PRODUCT_PID0XA007', export_intermediate=True)
```

4. **Step 3 (activation upgrade)**: if all of the above INT8 optimization measures fail, consider upgrading the activations to `INT16`.

```
amlnn.config(normalization_mean=[[128,128,128]],normalization_std=
[[128,128,128]], \
    quantized_dtype='w8a16', quantized_method='perchannel',
 activation_dtype='i16', \
    activation_quant_algo="histogram",weight_quant_algo="lsq", \
    target_platform='PRODUCT_PID0XA007', export_intermediate=True)
```

5. **Step 4 (activations not quantized):** when accuracy is still insufficient, switch the activations to the f16/bf16 non-quantized mode (**only supported on ADLA3**)

```
amlnn.config(normalization_mean=[[128,128,128]],normalization_std=
[[128,128,128]], \
    quantized_dtype='w8a16', quantized_method='perchannel',
 activation_dtype='f16', \
    activation_quant_algo="histogram", weight_quant_algo="lsq", \
    target_platform='PRODUCT_PID0XA007', export_intermediate=True)
 #or activation_dtype='bf16'
```

6. **Step5 (fully non-quantized):** if requirements are still not met, switch both weights and activations to the f16/bf16 non-quantized mode (**only supported on ADLA3**)

```
amlnn.config(normalization_mean=[[128,128,128]],normalization_std=
[[128,128,128]], \
    quantized_dtype='w16a16',activation_dtype='f16', \
    target_platform='PRODUCT_PID0XA007', export_intermediate=True)
 #or activation_dtype='bf16'
```

> **Core logic**: CV models are structurally simple and locally strong; mixed precision is usually unnecessary, and W8A8 fully-INT8 deployment can be achieved through careful PTQ (post-training quantization) parameter adjustment.

> **ADLA2 adaptation notes**: Step4 and Step5 are only supported on ADLA3 platforms. If, when executing up to Step3 on ADLA2, the accuracy still fails to meet the target, the INT16+Float mixed quantization scheme must be adopted, where Float operators run on the CPU.

2.4.2 Transformer Model Quantization Guidance

Transformer-type architectures such as LLM, ViT, and Swin Transformer rely on global attention mechanisms; the activation distribution exhibits a pronounced heavy-tail characteristic with many extreme outliers, and key operators such as LayerNorm and Softmax have extremely low tolerance to quantization error. Directly applying the traditional CNN fully-INT8/INT16 integer quantization scheme very easily leads to a cliff-like accuracy collapse. On ADLA2 platforms, a **mixed-precision quantization** strategy is recommended, with floating-point operators automatically scheduled to run on the CPU; on ADLA3 platforms, the native FP16/BF16 hardware acceleration feature can be enabled directly.

Recommended path:

ADLA2:

When deploying Transformer models on ADLA2 platforms, directly using fully-INT8/INT16 quantization means most models' accuracy targets cannot be met; therefore mixed-precision quantization (INT16+Float mix) is the mandatory solution on this architecture.

ADLA3:

- **Step 1 (activations not quantized w4a16)**: activations use the f16/bf16 non-quantized mode, weights use 4-bit.

```
amlnn.config(quantized_dtype='w4a16', quantized_method='perchannel',
 activation_dtype='f16', \
    weight_quant_algo="decoupleQ", target_platform='PRODUCT_PID0XA007',
export_intermediate=True)
 #or activation_dtype='bf16'
```

- **Step 2 (activations not quantized w8a16)**: activations use the f16/bf16 non-quantized mode, weights use 8-bit.

```
amlnn.config(quantized_dtype='w8a16', quantized_method='perchannel',
 activation_dtype='f16', \
    weight_quant_algo="lsq", target_platform='PRODUCT_PID0XA007',
export_intermediate=True)
#or activation_dtype='bf16'
```

- **Step3 (fully non-quantized)**: both weights and activations switch to the f16/bf16 non-quantized mode

```
amlnn.config(quantized_dtype='w16a16', activation_dtype='f16', \
    target_platform='PRODUCT_PID0XA007', export_intermediate=True)
#or activation_dtype='bf16'
```

2.4.3 Quantization Guidance Summary

Characteristic	CV Model (CNN)	Transformer Model (ViT/LLM)
Data distribution	Relatively uniform, strong local correlation	Highly non-uniform, with severe outliers
Sensitive operators	Few; Conv/FC are quite robust	Many; LayerNorm, Softmax, Attention are extremely sensitive
Preferred strategy	Full INT8 quantization (W8A8)	• **ADLA2:** mixed precision • **ADLA3:** use activations-not-quantized, or both weights and activations not quantized schemes
Tuning focus	Quantization granularity (Per-channel) and calibration algorithm	Layer sensitivity analysis, accuracy preservation of key layers
Recommended path	INT8 Per-layer → Per-channel → advanced calibration algorithms	w4a16 --> w8a16 --> w16a16
Alternative solutions	INT16 (only when INT8 fails completely)	w4a16/w8a16 multiple mixes

3 Quantization Conversion

3.1 Quantization Conversion Example

```
import os
from amlnn.api import AMLNN

def convert_and_quantize_model():
    """
    Load an ONNX model, configure quantization, compile, and export an ADLA model
using amlnn-toolkit.
    """
    # 1. Initialize the AMLNN instance
    # log_level: set the log level; INFO for viewing the normal flow, DEBUG for
troubleshooting details
    amlnn = AMLNN(log_level='INFO')

    # 2. Define paths and basic configuration
    model_path = 'resnet34-v1-7.onnx'
    dataset_path = './dataset.txt'
    output_dir = 'resnet34-v1-7_adla_unify'

    try:
        # 3. Load the ONNX model
        print(f"[Step 1] Loading ONNX model: {model_path}")
        amlnn.load_onnx(model=model_path)

        # 4. Configure quantization and compilation parameters
        print("[Step 2] Configuring quantization parameters...")

        # Description of core quantization parameters:
        # - quantized_dtype: 'w8a8' means both weights and activations use INT8
quantization, balancing accuracy and speed
        # - quantized_method: 'perchannel' perchannel quantization, usually more
accurate than per-tensor
        # - apply_smooth_o3: enable SmoothQuant and other optimization
techniques to reduce the impact of outliers on quantization
        # - normalization: set Mean/Std according to the model's training-time
preprocessing
        amlnn.config(
            normalization_mean=[[128, 128, 128]],
            normalization_std=[[128, 128, 128]],
            quantized_dtype='w8a8',          # Quantization type: INT8 weights +
INT8 activations
            quantized_method='perchannel',   # Quantization granularity:
perchannel
            apply_smooth_o3=True,            # Enable advanced smoothing
optimization
            target_platform='PRODUCT_PID0XA007', # Target hardware platform ID
            export_intermediate=True         # Export intermediate-layer model
for easy verification of quantization accuracy
        )
```

```
        # 5. Model compilation and quantization calibration
        # dataset: provide a representative dataset for PTQ (Post-Training
Quantization) calibration
        print(f"[Step 3] Compiling and calibrating with dataset:
{dataset_path}")
        amlnn.compile(dataset=dataset_path, iterations=200)

        # 6. Export the ADLA model
        print(f"[Step 4] Exporting ADLA model to directory: {output_dir}")
        amlnn.export_adla()

        print("[Success] Model conversion completed successfully.")

    except Exception as e:
        print(f"[Error] Failed to convert model: {str(e)}")
        raise

if __name__ == '__main__':
    convert_and_quantize_model()
```

> **Quantization Configuration and Data Preparation Guide**
>
> - **Parameter configuration strategy**: the core quantization parameters (such as quantization type, granularity, quantization algorithm, smoothing strategy, etc.) must be defined through the `config()` interface; the dynamic range statistics required for calibration, however, depend on the `compile()` interface
> - **Calibration dataset requirements**: please make sure the file pointed to by `./dataset.txt` contains **representative** input samples
>   - **Quantity recommendation**: usually **100~1000** images or npy datasets are sufficient to meet the calibration needs of PTQ (post-training quantization).
>   - **Quality recommendation**: the data distribution should cover the typical cases in the business scenario to avoid accuracy loss caused by data bias.

3.2 Model Format and Quantization Algorithm Compatibility

This chapter mainly describes the model storage formats supported by the different hardware platforms, as well as the Feature and Weights quantization algorithms supported during conversion of each format. For a more complete description of the quantization conversion parameters, please refer to the document <03_Amlogic_NPU_API_Reference_AMLNN>.

- **Summary of model storage format support across different hardware platforms:**
  - **Support Params**: defines the configuration parameters required for the corresponding Format.

ADLA	target_platform	Format (weights*Feature)	Support Params
ADLA2	PRODUCT_PID0XA001 ~ PRODUCT_PID0XA006	Int8*Int8	quantized_dtype='w8a8'
		Int8*Int16	quantized_dtype='w8a16', activation_dtype='i16'
ADLA3	PRODUCT_PID0XA007/PRODUCT_PID0XA008	Int4*int8	quantized_dtype='w4a8'

ADLA	target_platform	Format (weights*Feature)	Support Params
		Int8*Int8	quantized_dtype='w8a8'
		Int4*f16	quantized_dtype='w4a16' , activation_dtype='f16'
		Int4*bf16	quantized_dtype='w4a16' , activation_dtype='bf16'
		Int8*int16	quantized_dtype='w8a16', activation_dtype='i16'
		Int8*f16	quantized_dtype='w8a16', activation_dtype='f16'
		Int8*bf16	quantized_dtype='w8a16', activation_dtype='bf16'
		f16*f16	quantized_dtype='w16a16', activation_dtype='f16'
		bf16*bf16	quantized_dtype='w16a16', activation_dtype='bf16'

- **Summary of Feature and Weight quantization algorithms corresponding to each format:**

Format (weights*Feature)	quantized_dtype	activation_dtype	weight_quant_algo	activation_quant_algo
Int4*int8	w4a8	None	normal/decoupleQ/lsq	normal/histogram/omse/percentile
Int8*int8	w8a8	None	normal/lsq	normal/histogram/omse/percentile
Int4*f16	w4a16	f16	normal/decoupleQ/lsq	/
Int4*bf16	w4a16	bf16	normal/decoupleQ/lsq	/
Int8*int16	w8a16	i16	normal/lsq	normal/histogram/omse/percentile
Int8*f16	w8a16	f16	normal/lsq	/
Int8*bf16	w8a16	bf16	normal/lsq	/
f16*f16	w16a16	f16	/	/
bf16*bf16	w16a16	bf16	/	/

- **Description of the config interface parameters:**
  - **activation_quant_algo: activation quantization algorithm; valid values [normal, histogram, percentile, omse]**
    - **histogram algorithm parameters**
      - histogram_bins: the core configuration parameter of the histogram calibration algorithm. After taking the absolute value of the floating-point activations, the values are binned into intervals according to the specified number of bins; the quantization error of each clipping interval is computed in traversal, and finally the interval with the smallest error is selected, outputting the optimal quantization range max_value and min_value. **Default value: 2048**; valid values: [1024,2048,4096]
    - **percentile algorithm parameters**
      - percentile_alpha: the percentile clipping threshold; default 0.99999; valid range: (0~1). Directly specifies the upper quantile of quantization, automatically using `1-percentile_alpha` as the lower quantile, clipping the very small number of outliers at both ends and preventing the quantization range from being excessively stretched by extreme values. The larger the

value, the smaller the clipping range, and the closer the quantization range is to the original data distribution.

- percentile_sigma: exponential moving average smoothing coefficient; default 0.01; valid range: (0~1). Used to dynamically update statistics in multi-batch calibration scenarios, with the formula `val_new = (1-sigma)*val_old + sigma*val_current`. The smaller the value, the higher the weight of historical data, the lower the share of newly arriving single-batch data, and the stronger the statistical result's resistance to fluctuation.

- **omse algorithm parameters**
  - omse_range: defines the total number of candidate clipping intervals generated by the OMSE (Optimal Mean Square Error) algorithm. Based on this count, the algorithm generates multiple groups of candidate intervals with different degrees of shrinkage, computes the quantization MSE of each interval in turn, and finally selects the group with the smallest error as the optimal quantization clipping range. The larger the value, the finer the search space, but the computational overhead increases accordingly. **Default value: 90**; valid range: (0~100)
  - omse_step: defines the incremental proportion by which the upper and lower bounds of the quantization interval synchronously shrink inward in each iteration. While traversing the candidate intervals, each iteration adds one `omse_step` value to the shrink coefficient from the previous iteration, thereby progressively narrowing the quantization range to find the best clipping point with the minimum MSE. This parameter controls the granularity of the interval shrinkage and the search efficiency. **Default value: 0.02**; valid range: (0~1)

- **weight_quant_algo:** weight quantization algorithm; valid values [normal/decoupleQ/lsq]
- **apply_smooth_o3**: Smooth O3 smoothing optimization switch; default value `false`.
- **quantized_method**: weight quantization granularity; supports perchannel / perlayer; default perchannel. perchannel assigns quantization parameters per channel with higher accuracy; perlayer shares one set of parameters for the whole layer, and some models are prone to obvious accuracy drops.
- **inference_input_type**: specifies the input data type when exporting the model; default value `"quant"` (i.e. consistent with the quantization type).

  This parameter must be given a valid value according to the configuration of `quantized_dtype` and `activation_dtype`:

  - When `quantized_dtype` is `'w4a8'` or `'w8a8'`, supported values are `'int8'`, `'uint8'`, or `'float32'`.
  - When `quantized_dtype` is `'w8a16'` and `activation_dtype` is `'i16'`, supported values are `'int16'` or `'float32'`.
- **inference_output_type**: specifies the output data type of the exported ADLA model; default value `"quant"` (consistent with the quantization conversion type).
  - When `quantized_dtype` is `'w4a8'` / `'w8a8'`, supported configurations are `int8`, `uint8`, `float32`.
  - When `quantized_dtype` is `'w8a16'` and `activation_dtype` is `'i16'`, supported configurations are `int16`, `float32`.

- **export_intermediate**: controls whether to export the intermediate IR (intermediate representation) produced during conversion as a TFLite model file, used for verifying quantization accuracy. Default value `False (not exported)`.
- **group_size**: group granularity for 4-bit quantization of FC operators; takes effect only when `quantized_dtype` is `w4a8` or `w4a16`. Supported values: 32 / 64 / 128. Smaller values give higher accuracy; larger values give higher inference efficiency.
- **hybrid_ratio**: takes effect only in `w4a8` and `w4a16` quantization modes; controls the proportion of convolution operators quantized with 4-bit weights. The larger the value, the higher the quantization compression ratio and the lower the memory footprint, but the risk of accuracy loss also increases accordingly.

- **Description of the compile interface parameters:**
  - **dataset**: path of the quantization calibration dataset configuration file. The file must list the image or `.npy` data paths used for computing activation value distribution statistics.
    - **Default value**: `None` (quantization is performed using virtual data generated from random numbers).
    - **Note**: the default mode is only suitable for verifying the conversion flow or testing inference performance; because it lacks real data distribution statistics, **model accuracy cannot be guaranteed**. For formal deployment, please be sure to provide representative real calibration data.
  - **iterations**: number of samples used for quantization calibration, read from the dataset specified in `dataset.txt`.
    - **Default value**: `-1` (traverse all samples in the dataset).
    - **Special behavior**: if no dataset is configured (`dataset` is `None`), this parameter is ignored and only a single group of random data is generated for quantization (only for flow testing; no practical accuracy meaning).

3.3 Model Accuracy Testing Under Different Quantization Granularities

When all models use the baseline scheme of **w8a8 + perchannel + normal calibration + normal weight quantization**, the Top1 accuracy loss is kept within **3 percentage points**; ResNet50, ViT-B/16, and MobileNetV2 perform better, with baseline accuracy loss below **1.5 percentage points**.

On this basis, by combining activation calibration algorithms, LSQ weight quantization, and SmoothO3 smoothing optimization strategies, the accuracy loss of the optimal quantization scheme for all models is further compressed to within **1.5 percentage points**; MNASNet1_3 loses only 0.4 percentage points, and ResNet50 even surpasses the floating-point baseline after quantization, with overall quantization optimization gains being significant

Model Name	SmoothO3	Quantization Granularity	Feature Calibration Algorithm	Weight Quantization Algorithm	Top1 Accuracy After Quantization	Floating-Point Baseline Top1 Accuracy	Accuracy Loss Rate	Remarks
EfficientNet-B7	❌	perchannel	normal	normal	0.7145	0.7300	-1.55%	Baseline quantization scheme
	❌	perchannel	percentile	lsq	**0.7175**	0.7300	-1.25%	Accuracy-optimal configuration

Model Name	SmoothO3	Quantization Granularity	Feature Calibration Algorithm	Weight Quantization Algorithm	Top1 Accuracy After Quantization	Floating-Point Baseline Top1 Accuracy	Accuracy Loss Rate	Remarks
	✅	perchannel	percentile	normal	0.7175	0.7300	-1.25%	Smoothing optimization enabled
MNASNet1_3	❌	perchannel	normal	normal	0.7395	0.7590	-1.95%	Baseline quantization scheme
	❌	perchannel	percentile	lsq	**0.7550**	0.7590	-0.40%	Accuracy-optimal configuration
MobileNetV2	❌	perchannel	normal	normal	0.7080	0.7215	-1.35%	Baseline quantization scheme
	❌	perchannel	percentile	normal	**0.7150**	0.7215	-0.65%	Accuracy-optimal configuration
RegNetX-16GF	❌	perchannel	normal	normal	0.7605	0.7900	-2.95%	Baseline quantization scheme
	❌	perchannel	histogram	lsq	**0.7760**	0.7900	-1.40%	Accuracy-optimal configuration
	✅	perchannel	histogram	normal	0.7760	0.7900	-1.40%	Smoothing optimization enabled
RegNetY-16GF	❌	perchannel	normal	normal	0.7650	0.7900	-2.50%	Baseline quantization scheme
	❌	perchannel	histogram	lsq	**0.7770**	0.7900	-1.30%	Accuracy-optimal configuration
ResNet50	❌	perchannel	normal	normal	0.7950	0.7995	-0.45%	Baseline quantization scheme
	❌	perchannel	histogram	lsq	**0.8000**	0.7995	+0.05%	Accuracy surpasses floating-point after quantization
ShuffleNetV2	❌	perchannel	normal	normal	0.7345	0.7600	-2.55%	Baseline quantization scheme
	❌	perchannel	percentile	lsq	**0.7480**	0.7600	-1.20%	Accuracy-optimal configuration
ViT-B/16	❌	perchannel	normal	normal	0.7845	0.7980	-1.35%	Baseline quantization scheme
	❌	perchannel	percentile	normal	**0.7860**	0.7980	-1.20%	Accuracy-optimal configuration

With the normal+normal baseline quantization at perlayer granularity, accuracy loss is generally far higher than with the perchannel scheme; most models lose more than **3 percentage points**, and the baseline loss of ShuffleNetV2 and ViT-B/16 exceeds **13 percentage points** — the basic accuracy shortcoming of perlayer quantization is very prominent.

By combining feature calibration algorithms, LSQ weight quantization, and SmoothO3 smoothing optimization strategies, the accuracy loss of the optimal scheme for all models is greatly narrowed: the optimal loss of ResNet50 and RegNetY-16GF is kept within **2 percentage points**; ViT-B/16 shows the most significant optimization benefit, with its baseline loss of 13.6 percentage points directly compressed to 1.25 percentage points, completely erasing the basic

accuracy gap versus perchannel quantization. Meanwhile SmoothO3 delivers prominent gains in perlayer quantization, with 4 of the 8 tested models enabling this optimization in their optimal configurations.

Model Name	SmoothO3	Quantization Granularity	Feature Calibration Algorithm	Weight Quantization Algorithm	Top1 Accuracy After Quantization	Floating-Point Baseline Top1 Accuracy	Accuracy Loss Rate	Remarks
EfficientNet-B7	❌	perlayer	normal	normal	0.6770	0.7300	-5.30%	Baseline quantization scheme
	❌	perlayer	percentile	lsq	**0.7150**	0.7300	-1.50%	Accuracy-optimal configuration
MNASNet1_3	❌	perlayer	normal	normal	0.7150	0.7590	-4.40%	Baseline quantization scheme
	✅	perlayer	omse	lsq	**0.7380**	0.7590	-2.10%	Accuracy-optimal configuration
MobileNetV2	❌	perlayer	normal	normal	0.6680	0.7215	-5.35%	Baseline quantization scheme
	❌	perlayer	percentile	lsq	**0.6800**	0.7215	-4.15%	Accuracy-optimal configuration
RegNetX-16GF	❌	perlayer	normal	normal	0.7500	0.7900	-4.00%	Baseline quantization scheme
	✅	perlayer	histogram	normal	**0.7715**	0.7900	-1.85%	Accuracy-optimal configuration
RegNetY-16GF	❌	perlayer	normal	normal	0.7605	0.7900	-2.95%	Baseline quantization scheme
	❌	perlayer	histogram	normal	**0.7785**	0.7900	-1.15%	Accuracy-optimal configuration
ResNet50	❌	perlayer	normal	normal	0.7815	0.7995	-1.80%	Baseline quantization scheme
	✅	perlayer	histogram	normal	**0.7940**	0.7995	-0.55%	Accuracy-optimal configuration
ShuffleNetV2	❌	perlayer	normal	normal	0.6220	0.7600	-13.80%	Baseline quantization scheme
	✅	perlayer	histogram	lsq	**0.6665**	0.7600	-9.35%	Accuracy-optimal configuration
ViT-B/16	❌	perlayer	normal	normal	0.6620	0.7980	-13.60%	Baseline quantization scheme
	❌	perlayer	percentile	lsq	**0.7855**	0.7980	-1.25%	Accuracy-optimal configuration

4 Hybrid Quantization Guidance

The content of this chapter applies to **all Amlogic NPU platforms covered by this document**.

4.1 Introduction to Mixed Quantization

The `amlnn-toolkit` provides multiple mixed-precision quantization schemes that strike a balance between inference performance and model accuracy. The supported scenarios include:

- **INT8 + INT16 Mixed Integer Quantization**: An all-integer combination that balances performance and accuracy.
- **INT8 + Float Mixed Quantization**: Sensitive layers retain floating-point precision while the remaining layers use INT8 acceleration.
- **INT16 + Float Mixed Quantization**: Suitable for models with high requirements on the dynamic range of activation values.
- **INT8 + INT16 + Float Mixed Quantization**: A three-level precision combination, serving as the fallback scheme for accuracy-priority scenarios.

Mixed quantization can be configured in two ways: based on operator name and based on operator category.

- **Configuration Based on Operator Category**: No need to export the quantized TFLite model in advance; simply edit the `quant_config.json` file to complete the configuration.
- **Configuration Based on Operator Name**: The quantized TFLite model must be exported first, and then the mixed quantization objects are specified one by one according to the operator names (Output Name) in the model.

> The model conversion stage already supports mixed quantization configurations with multiple combinations. Note that in the process of finally generating the ADLA model and running the ADLA model, some scenarios may still have exceptions, and the related capabilities need further verification and improvement.

4.2 Mixed Quantization Guidelines

By combining 8-bit and 16-bit integer representations, storage and computation overhead is reduced while the high-precision characteristics of key operators are preserved.

Step1: Export the quantized TFLite model

If the mixed quantization configuration method based on operator name is adopted, the quantized TFLite model must be exported in advance. During export, set `config(export_intermediate=True)` in the configuration to generate a quantized TFLite model containing intermediate layer information, so that the configuration objects can be located by operator name later.

Step2: Mixed Quantization Configuration Instructions

- **INT8 Quantization Base Configuration File**:

  The default JSON configuration file for INT8 conversion with `amlnn-toolkit` is as follows:

```
[
  {
    "regex": ".*",
```

```
    "operation": "*",
    "algorithm_key": "min_max_uniform_quantize",
    "op_config": {
      "activation_tensor_config": {
        "num_bits": 8,
        "symmetric": false,
        "granularity": "TENSORWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "weight_tensor_config": {
        "num_bits": 8,
        "symmetric": true,
        "granularity": "CHANNELWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "compute_precision": "INTEGER",
      "explicit_dequantize": false,
      "skip_checks": true,
      "min_weight_elements": 0
    }
  }
]
```

- **INT16 Quantization Base Configuration File**

```
[
  {
    "regex": ".*",
    "operation": "*",
    "algorithm_key": "min_max_uniform_quantize",
    "op_config": {
      "activation_tensor_config": {
        "num_bits": 16,
        "symmetric": true,
        "granularity": "TENSORWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "weight_tensor_config": {
        "num_bits": 8,
        "symmetric": true,
        "granularity": "CHANNELWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "compute_precision": "INTEGER",
      "explicit_dequantize": false,
      "skip_checks": true,
      "min_weight_elements": 0
    }
  }
]
```

- **Batch Configuration Based on Operator Type (Operation Type)**

  Suitable for scenarios where the precision of a certain category of common operators needs to be improved uniformly.

  - **Configuration Basis**: Use the operator's operation type (`operation`, such as `Sub`, `ADD`, `Conv2D`, etc.) as the identifier. Configure via `operation`.
  - **INT16 Batch Configuration Based on Operator Type**

```
{
  "regex": ".*",
  "operation": "SUB",
  "algorithm_key": "min_max_uniform_quantize",
  "op_config": {
    "activation_tensor_config": {
      "num_bits": 16,
      "symmetric": true,
      "granularity": "TENSORWISE",
      "dtype": "INT",
      "block_size": 0
    },
    "weight_tensor_config": {
      "num_bits": 8,
      "symmetric": true,
      "granularity": "CHANNELWISE",
      "dtype": "INT",
      "block_size": 0
    },
    "compute_precision": "INTEGER",
    "explicit_dequantize": false,
    "skip_checks": true,
    "min_weight_elements": 0
  }
}
```

  - **Float Batch Configuration Based on Operator Type**

```
{
  "regex": ".*",
  "operation": "DIV",
  "algorithm_key": "no_quantize"
}
```

- **Configuration Based on Operator Name (Output Name)**

  - **Configuration Basis**: Use the operator's output node name (`outputname`) as the unique identifier. Configure via the `regex` parameter.
  - **INT16 Configuration Based on Operator Name (Output Name)**

```
{
  "regex": "PartitionedCall_1:0",
  "operation": "*",
  "algorithm_key": "min_max_uniform_quantize",
  "op_config": {
    "activation_tensor_config": {
```

```
        "num_bits": 16,
        "symmetric": true,
        "granularity": "TENSORWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "weight_tensor_config": {
        "num_bits": 8,
        "symmetric": true,
        "granularity": "CHANNELWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "compute_precision": "INTEGER",
      "explicit_dequantize": false,
      "skip_checks": true,
      "min_weight_elements": 0
    }
  }
```

- **Float Configuration Based on Operator Name (Output Name)**

```
  {
    "regex":  "PartitionedCall_1:0",
    "operation": "*",
    "algorithm_key": "no_quantize"
  }
```

Step3: Edit the Mixed Quantization Configuration File

- **INT8+INT16 Mixed Quantization Configuration File**

  After using the configuration methods based on operator name and based on operator category respectively, the generated `quant_config.json` configuration file is as follows.

```
  [
    {
      "regex": ".*",
      "operation": "*",
      "algorithm_key": "min_max_uniform_quantize",
      "op_config": {
        "activation_tensor_config": {
          "num_bits": 8,
          "symmetric": false,
          "granularity": "TENSORWISE",
          "dtype": "INT",
          "block_size": 0
        },
        "weight_tensor_config": {
          "num_bits": 8,
          "symmetric": true,
          "granularity": "CHANNELWISE",
          "dtype": "INT",
          "block_size": 0
```

```
      },
      "compute_precision": "INTEGER",
      "explicit_dequantize": false,
      "skip_checks": true,
      "min_weight_elements": 0
    }
  },
   {
    "regex": ".*",
    "operation": "SUB",
    "algorithm_key": "min_max_uniform_quantize",
    "op_config": {
      "activation_tensor_config": {
        "num_bits": 16,
        "symmetric": true,
        "granularity": "TENSORWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "weight_tensor_config": {
        "num_bits": 8,
        "symmetric": true,
        "granularity": "CHANNELWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "compute_precision": "INTEGER",
      "explicit_dequantize": false,
      "skip_checks": true,
      "min_weight_elements": 0
    }
  },
  {
    "regex": "PartitionedCall_1:0",
    "operation": "*",
    "algorithm_key": "min_max_uniform_quantize",
    "op_config": {
      "activation_tensor_config": {
        "num_bits": 16,
        "symmetric": true,
        "granularity": "TENSORWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "weight_tensor_config": {
        "num_bits": 8,
        "symmetric": true,
        "granularity": "CHANNELWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "compute_precision": "INTEGER",
      "explicit_dequantize": false,
      "skip_checks": true,
      "min_weight_elements": 0
```

```
    }
  }
]
```

- **INT8+Float Mixed Quantization Configuration File**

  After using the configuration methods based on operator name and based on operator category respectively, the generated `quant_config.json` configuration file is as follows.

```
[
  {
    "regex": ".*",
    "operation": "*",
    "algorithm_key": "min_max_uniform_quantize",
    "op_config": {
      "activation_tensor_config": {
        "num_bits": 8,
        "symmetric": false,
        "granularity": "TENSORWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "weight_tensor_config": {
        "num_bits": 8,
        "symmetric": true,
        "granularity": "CHANNELWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "compute_precision": "INTEGER",
      "explicit_dequantize": false,
      "skip_checks": true,
      "min_weight_elements": 0
    }
  },
  {
    "regex": ".*",
    "operation": "DIV",
    "algorithm_key": "no_quantize"
   },
   {
    "regex":  "PartitionedCall_1:0",
    "operation": "*",
    "algorithm_key": "no_quantize"
  },
]
```

- **INT16+Float Mixed Quantization Configuration File**

  After using the configuration methods based on operator name and based on operator category respectively, the generated `quant_config.json` configuration file is as follows.

```
[
```

```
   {
    "regex": ".*",
    "operation": "*",
    "algorithm_key": "min_max_uniform_quantize",
    "op_config": {
      "activation_tensor_config": {
        "num_bits": 16,
        "symmetric": true,
        "granularity": "TENSORWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "weight_tensor_config": {
        "num_bits": 8,
        "symmetric": true,
        "granularity": "CHANNELWISE",
        "dtype": "INT",
        "block_size": 0
      },
      "compute_precision": "INTEGER",
      "explicit_dequantize": false,
      "skip_checks": true,
      "min_weight_elements": 0
    }
  },
  {
    "regex": ".*",
    "operation": "DIV",
    "algorithm_key": "no_quantize"
   },
   {
    "regex":  "PartitionedCall_1:0",
    "operation": "*",
    "algorithm_key": "no_quantize"
  }
]
```

Step4: Execute Mixed Quantization

Before executing the conversion, you must first specify the path of the modified configuration file through an environment variable so that the tool loads the custom quantization configuration:

```
export AMLNN_SET_QUANT_CONFIG_FILE='Your_dir/quant_config.json'
```

Once the configuration is complete, run the conversion script for it to take effect.

5 quantize_optimization_tool Usage Guide

This chapter applies to **all Amlogic NPU platforms covered by this document**. `quantize_optimization_tool` is an automatic parameter search tool. By traversing the activation quantization algorithms and their hyperparameter space and computing the output similarity between the quantized model and the floating-point model under each configuration, it helps the user select the optimal parameter combination with the smallest accuracy loss (highest similarity). Download link: https://github.com/Amlogic-NN/amlnn-toolkit/tree/main/amlnn_toolkit/function_tool/03_quantize_optimization_tool

5.1 Tool Introduction

- **The tool's parameter configuration is as follows:**

```
def __init__(self):
    self.mdoel_type='pt2'
    self.model_path='/xxxxx/resnet50_exported.pt2'
    self.source_file='/xxxxxx/ILSVRC2012_val_256.txt'
    self.normalization_mean = [[123.675, 116.28, 103.53]]
    self.normalization_std =  [[58.395, 57.12, 57.375]]
    self.iteration=32
    self.outdir='./'
    self.input_shape=[224,224,3]
    self.input_npy_path=['/xxxx/resnet50_nhwc.npy']
    self.output_npy_path=['/xxxx/resnet50_out_0.npy']
    self.activation_quant_algo=["normal",'percentile',"histogram","omse"]
    self.weight_quant_algo =["normal","lsq"]
    self.inference_input_type="float32"
    self.inference_output_type="float32"
    self.percentile_sigma =[0.02,0.04,0.06,0.08,0.1,0.12,0.14,0.16,0.18,0.2]
    self.percentile_alpha =[0.9999,0.99999,0.999999,0.9999999]
    self.omse_range =[80,85,90,95,98]
    self.omse_step =[0.01,0.012,0.014,0.016,0.018,0.02]
    self.num_histogram_bins = [2048, 4096, 8192 , 12288,16384]
    os.environ['AMLNN_ENABLE_CHANNEL_LAST_IO']=str(True)
    self.result="test_result.txt"
    self.output=[]
    self.euclidean_dis=[]
    self.cos_sim=[]
```

- **Parameter description:**
  - **Basic configuration items**
    - `model_type`: model format selection; supports `pt2`, `tflite`; when `pt2` is selected, the tool automatically completes the prerequisite format conversion
    - `model_path`: local path of the floating-point model; fill in the pt2 model or tflite file path according to `model_type`
    - `input_shape`: explicitly specifies the dimension structure of the model's input tensor
    - `outdir`: unified output directory for quantized models, search logs, and result files

        - `iteration`: actual number of quantization calibration samples read from the calibration set
    - **Data configuration items**
        - `source_file`: path of the calibration set index file, i.e. the pre-prepared txt file containing the image/npy paths
        - `normalization_mean`, `normalization_std`: the normalization mean and variance of the input calibration set; must be fully aligned with the original floating-point model's preprocessing logic
        - `input_npy_path`, `output_npy_path`: lists of npy file paths for the floating-point model's reference inputs and outputs, used to compute similarity
    - **Quantization search configuration items**
        - `activation_quant_algo`: selectable range of activation quantization algorithms; supports `normal`, `percentile`, `histogram`, `omse`
        - `weight_quant_algo`: selectable range of weight quantization algorithms; supports `normal`, `lsq`
        - `percentile_sigma`, `percentile_alpha`: hyperparameter search range of the percentile algorithm
        - `omse_range`, `omse_step`: hyperparameter range and traversal step of the omse algorithm
        - `num_histogram_bins`: number of bins of the histogram algorithm; normally set to 2048 by default, and can also be set to a multiple of 2048 to expand the search range
    - **Special adaptation items**
        - `inference_input_type`, `inference_output_type`: specify whether the inputs and outputs at the inference stage retain floating-point types; the quantization of the first and last layers can be disabled as needed
        - If the model input is in NCHW format, add the environment variable configuration at the beginning of the script: `os.environ['AMLNN_ENABLE_CHANNEL_LAST_IO'] = 'false'`

5.2 Usage Guide

- **Preliminary preparation**: three types of core resources must be prepared in advance before use to avoid interruption of the search flow:
    - **Calibration dataset index file**: in `.txt` format, with one image path or one `.npy` data file path per line
    - **Model preprocessing parameters**: the normalization mean and normalization standard deviation corresponding to the input calibration set
    - **Floating-point model reference data**: the input `.npy` list and the corresponding output `.npy` list used for accuracy comparison
- **Tool search flow**: no need to manually debug and verify group by group; the tool automatically completes the full-chain operations:
    - Traverses all activation quantization algorithms within the preset range, while simultaneously traversing the `normal` and `lsq` modes of weight quantization
    - Generates the corresponding quantized model for each quantization configuration and computes its output similarity with the floating-point reference model
    - All search results are sorted by similarity and written uniformly to `test_result.txt` in the output directory; simply take the top-ranked parameters as the final deployment

scheme

5.3 Usage Recommendations

1. For initial testing, it is recommended to first narrow the hyperparameter search range, quickly complete the verification flow, and only open up the full search after confirming the results are correct
2. The calibration dataset must cover the typical input scenarios of the model's actual business; otherwise deviations such as "high similarity but poor online accuracy" will occur
3. After the search is complete, prefer the configuration with similarity Top1 in `test_result.txt`; only manually fine-tune local parameters when specific operators drop in accuracy

5.4 Custom Evaluation Criteria

The tool currently has a built-in evaluation criterion based on the cosine similarity of single-frame data: during the parameter search process, it traverses all parameter combinations and selects the one with the highest cosine similarity as the final quantization parameters. However, for some models, single-frame similarity cannot accurately reflect the true level of accuracy. Therefore, it is necessary to introduce the capability for customers to define custom evaluation criteria, whereby the customer specifies the accuracy evaluation metric, and the tool then performs the parameter search and selects the optimal value according to that criterion.

- **Current Built-in Evaluation Criterion**
  - Refer to the test_result interface in quantize_optimization_tool.py

```
def test_result(self, cmd_line):
    self.run_tflite()  # Run the exported quantized TFLite model

    self.compute_euclidean_dis_and_cos()  # Compare the quantized TFLite
results with the float model results

    self.record_result(cmd_line)  # Record the conversion parameters and the
comparison data between the quantized and float results
```

- **Modification Guidelines**
  - **run_tflite**: Implement the custom evaluation criterion within this method, run the quantized TFLite model, and output the accuracy results on the specified dataset.
  - **record_result**: Record the parameter configuration used in this conversion and the corresponding accuracy results, to facilitate subsequent parameter comparison and traceability.

```
def test_result(self, cmd_line):
    self.run_tflite()  # Run the exported quantized TFLite model and compute
accuracy metrics on the specified dataset according to a custom evaluation
criterion (e.g., Top1 accuracy, mAP, etc.)

    self.record_result(cmd_line)  # Record the conversion parameters and the
corresponding accuracy results
```

6 hybrid_ratio_tool Usage Guide

This chapter applies only to the **Amlogic ADLA3 platforms covered by this document**. It is designed specifically for the w4a16 and w4a8 quantization scenarios, with the core goal of finding the optimal balance between low-bit compression ratio and model accuracy. The tool precisely controls the proportion of 4-bit quantized Conv2D operators through the `hybrid_ratio` parameter

- The value range is 0~1: when `hybrid_ratio = 0`, all Conv2D operators are quantized with INT8; when `hybrid_ratio = 1`, all Conv2D operators are quantized with INT4.
- The tool automatically traverses the preset `hybrid_ratio` search interval and simultaneously outputs two core accuracy metrics — the cosine similarity and Euclidean distance between the quantized model and the floating-point model for each configuration — so that the user, considering the business's dual requirements for accuracy and performance, can independently select the final optimal quantization parameter combination.

Download link: https://github.com/Amlogic-NN/amlnn-toolkit/tree/main/amlnn_toolkit/function_tool/04_hybrid_ratio_tool

6.1 Tool Introduction

The tool's parameter configuration is as follows:

```
def __init__(self):

    self.model_type='pt2' #[pt2/tflite]
    self.trunk_name ="cat_16"
    self.model_path='/xxxx/yolovx-s.pt2'
    self.source_file='/xxxx/dataset.txt'
    self.normalization_mean = [[0.,0.,0.]]
    self.normalization_std =  [[255.0,255.0,255.0]]
    self.quantized_dtype="w4a16" #["w4a8","w4a16"]
    self.hybrid_ratio_all =np.round(np.linspace(0.1,0.5,2),2)
    self.groups=[32]
    self.iteration=32
    self.outdir='./'
    self.input_npy_path=['yolov8n_trunc_nhwc.npy']
    self.output_npy_path=['yolov8n_trunc_nhwc_out_0.npy']
    self.activation_quant_algo=["normal",'percentile',"histogram","omse"]
    self.weight_quant_algo =["lsq","decoupleQ"]
    self.inference_input_type="float32"
    self.inference_output_type="float32"

    self.percentile_sigma =[0.02,0.04,0.06,0.08,0.1,0.12,0.14,0.16,0.18,0.2]
    self.percentile_alpha =[0.9999,0.99999,0.999999,0.9999999]
    self.omse_range =[80,85,90,95,98]
    self.omse_step =[0.01,0.012,0.014,0.016,0.018,0.02]
    self.num_histogram_bins = [2048, 4096, 8192 , 12288,16384]
    os.environ['AMLNN_ENABLE_CHANNEL_LAST_IO']=str(True)
    self.result="test_result.txt"
    self.output=[]
    self.euclidean_dis=[]
```

```
    self.cos_sim=[]
```

Parameter description:

- **model_type**: supports two model input formats, `pt2` and `tflite`; if the input is a pt2 model, the output node name can be specified via `trunk_name` to truncate the model.
- **quantized_dtype**: quantization precision mode; options are `w4a16` and `w4a8`. In the w4a8 scenario, the lsq algorithm is enabled by default for weight quantization with no additional configuration required; in the w4a16 scenario, either of the two weight quantization algorithms lsq or decoupleQ can be manually selected.
- **hybrid_ratio_all**: the traversal search interval of the hybrid quantization ratio, with the value range limited to 0~1; the tool will automatically generate multiple ratio combinations within this interval to complete full validation.
- **groups**: the number of groups for block_wise group quantization of fully-connected (fc) operators; supports configuration as `[32, 64, 128]`; when this configuration is enabled, the fc operators' activations skip quantization and retain high precision.
- **weight_quant_algo**: takes effect only in the w4a16 quantization scenario; used to specify the weight quantization algorithm; options are `lsq` and `decoupleQ`.
- **activation_quant_algo**: takes effect only in the w4a8 quantization scenario; supports traversing the full parameter space of activation quantization algorithms and automatically locks onto the optimal activation quantization configuration.

6.2 YOLOX-S Model Validation

6.2.1 W4A16 Quantization Validation

This experiment uses the YOLOX-S floating-point model (mAP50=0.593, mAP50-95=0.405) as the baseline, validating the accuracy performance of the two weight quantization algorithms decoupleQ and LSQ under different hybrid precisions in the w4a16 scenario. The INT4 convolution layer proportion (features not quantized) is controlled via the `hybrid_ratio` parameter (0.0 = fully-INT8 quantization, 1.0 = fully-INT4 quantization); the higher the value, the greater the share of low-bit quantization.

- **Validation results**

hybrid_ratio	weight_quant_algo	map50	map50_95
0.0	decoupleQ	0.592	0.404
0.0	lsq	0.559	0.376
0.1	decoupleQ	0.591	0.401
0.1	lsq	0.548	0.362
0.2	decoupleQ	0.586	0.395
0.2	lsq	0.541	0.352
0.3	decoupleQ	0.582	0.389
0.3	lsq	0.528	0.337
0.4	decoupleQ	0.58	0.383

hybrid_ratio	weight_quant_algo	map50	map50_95
0.4	lsq	0.524	0.323
0.5	decoupleQ	0.577	0.377
0.5	lsq	0.512	0.308
0.6	decoupleQ	0.574	0.374
0.6	lsq	0.501	0.299
0.7	decoupleQ	0.563	0.366
0.7	lsq	0.486	0.291
0.8	decoupleQ	0.561	0.363
0.8	lsq	0.48	0.283
0.9	decoupleQ	0.54	0.347
0.9	lsq	0.434	0.252
1.0	decoupleQ	0.532	0.331
1.0	lsq	0.446	0.254

- **Analysis of results:**
  - **Fully-INT8 quantization (hybrid_ratio=0.0)**: decoupleQ's accuracy is almost on par with the floating-point model; mAP50 and mAP50-95 each drop by only 0.001 with virtually no loss; LSQ's two metrics drop by 0.034 and 0.029 respectively, already showing noticeable accuracy decay
  - **Low-to-medium hybrid quantization range (0.1~0.5)**: decoupleQ's accuracy declines gently and uniformly, with only a small loss at hybrid_ratio=0.5; LSQ's decay accelerates continuously, and its accuracy loss at the same ratio is far higher than decoupleQ's
  - **High-ratio INT4 range (0.6~1.0)**: the accuracy gap widens further; in the fully-INT4 state decoupleQ still maintains considerable detection capability with mAP50=0.532 and mAP50-95=0.331; LSQ's accuracy plunges drastically, with mAP50 only 0.446 and mAP50-95 as low as 0.254

6.2.2 w4a8 Quantization Validation

This experiment uses the YOLOX-S floating-point model (mAP50=0.593, mAP50-95=0.405) as the baseline, testing the accuracy performance of the two weight quantization algorithms under different `hybrid_ratio` proportions in the W4A8 quantization scenario.

- **Validation results**

hybrid_ratio	weight_quant_algo	map50	map50_95
0.0	decoupleQ	0.58	0.387
0.0	lsq	0.404	0.255
0.1	decoupleQ	0.579	0.387

hybrid_ratio	weight_quant_algo	map50	map50_95
0.1	lsq	0.383	0.238
0.2	decoupleQ	0.578	0.385
0.2	lsq	0.377	0.233
0.3	decoupleQ	0.576	0.383
0.3	lsq	0.36	0.218
0.4	decoupleQ	0.573	0.377
0.4	lsq	0.356	0.209
0.5	decoupleQ	0.572	0.374
0.5	lsq	0.346	0.201
0.6	decoupleQ	0.57	0.372
0.6	lsq	0.335	0.194
0.7	decoupleQ	0.567	0.37
0.7	lsq	0.32	0.185
0.8	decoupleQ	0.564	0.368
0.8	lsq	0.324	0.188
0.9	decoupleQ	0.559	0.365
0.9	lsq	0.285	0.163
1.0	decoupleQ	0.556	0.354
1.0	lsq	0.462	0.268

- **Analysis of results:**
  - **Overall pattern**: as `hybrid_ratio` rises and the proportion of 4-bit weight convolution layers increases, both algorithms' mAP50 and mAP50-95 accuracy gradually decline, and at the fully-quantized ratio **all of decoupleQ's accuracy metrics are higher than LSQ's**.
  - Pure 8-bit weight quantization (`hybrid_ratio=0.0`): decoupleQ's mAP50=0.58 and mAP50-95=0.387, with extremely small overall accuracy loss.
  - Fully 4-bit weight quantization (`hybrid_ratio=1.0`): decoupleQ still maintains considerable usable accuracy, with mAP50=0.556 and mAP50-95=0.354.

6.3 Usage Recommendations

- **Applicable scenarios**: in W4A8 or W4A16 quantization deployments, if fully-INT4 weight quantization (`hybrid_ratio=1.0`) results in accuracy failing to meet the target, please enable this tool for mixed-precision tuning.

- **Operation strategy**: use the tool to traverse the `hybrid_ratio` search space and automatically evaluate the accuracy performance under different hybrid ratios, thereby locking onto the optimal balance between model compression ratio and inference accuracy.
- **Parameter selection:**
    - **Weight Quant Algo**: the **decoupleQ** algorithm is strongly recommended as the first choice, due to its superior accuracy preservation capability under low-bit quantization.
    - **Hybrid Ratio**: based on the accuracy-compression ratio curve output by the tool, select the smallest INT4 proportion that satisfies the business accuracy threshold as the final conversion parameter.